DSA JOURNEY

Personal Learning Analysis

Day 1 through Day 100

About This Document

This document is a complete reflective analysis of a 100-day personal DSA journey — from the very first day through Day 100. It captures learning patterns, psychological shifts, recurring fears, major breakthroughs, strengths, weaknesses, and the gradual transformation from a beginner to someone with genuine algorithmic intuition. This is not a list of topics. It is a record of how thinking changed.

01 PHASE ONE
Days 1–60 · Foundations & Trees

02 PHASE TWO
Days 60–100 · Graphs & Internalization

03 FINAL ASSESSMENT
Day 100 · Where You Stand

PHASE ONE

Days 1 – 60

Foundations, Trees & Binary Search Trees

INTRODUCTION

60 Days of DSA — Reflective Journey Analysis

This document is a reflective analysis of your DSA journey from Day 1 to roughly Day 60. The purpose of this file is not just to list what topics you completed. It is meant to capture:

- How your thinking evolved
- How your fears changed
- How your learning style became clearer
- What you struggled with
- What became easier over time
- What your habits looked like
- What psychological battles happened alongside the technical growth

What makes this journey interesting is not simply that you solved problems. It is that over time, you slowly transformed from someone intimidated by DSA into someone who began recognizing patterns, reconstructing logic, debugging independently, and emotionally surviving difficult learning phases.

This was not a clean upward graph. There were periods of excitement, periods of exhaustion, periods of doubt, periods of feeling like a fraud, periods where concepts suddenly clicked, and periods where you feared you were only memorizing.

> *But through all of it, one thing remained surprisingly stable: You kept showing up.*

CORE LEARNING PROCESS

Your Core Learning Process

Over these 60 days, a very clear learning loop emerged. Your process usually looked like this:

1. Watch the lecture
2. Take screenshots or notes
3. Write handwritten notes

4. Explain the concept back to yourself
5. Attempt the LeetCode problem
6. Debug syntax and logic mistakes
7. Dry run the solution later
8. Revisit it during class revision

This became your primary engine for learning. You were never someone who learned comfortably through blind memorization. Whenever you tried to merely copy or mechanically remember code, your confidence immediately dropped.

You needed:

- Intuition
- Structure
- Pattern recognition
- Mental visualization
- Execution flow understanding

Only after these clicked did the implementation begin feeling natural.

EARLY PHASE

Your Initial Relationship With DSA

In the beginning, DSA often felt intimidating and abstract. There was a recurring feeling that:

> *"Everyone else probably understands this naturally, but I'm struggling to hold all these moving parts together."*

You especially struggled with: execution order, syntax-heavy implementations, STL usage, recursion flow, queue/stack mechanics, and mentally tracking pointers and traversal.

At the start, many problems felt completely isolated. Each algorithm seemed like a new universe, a new syntax system, a new mental model. You did not yet recognize reusable patterns. This made learning feel mentally expensive.

KEY BREAKTHROUGH

One of Your Earliest Big Realizations

One major conceptual breakthrough happened when you realized:

> *In recursive tree problems, you only solve for one node.*

This idea changed how you viewed trees. Before this realization, trees looked overwhelming — there were too many nodes mentally, the structure felt infinite, and you didn't know where to "hold" the problem.

Afterward, recursion became psychologically lighter. The tree no longer felt gigantic. You began trusting recursion to handle the rest. This single shift heavily influenced your confidence in recursive tree problems.

BINARY SEARCH PHASE

The Binary Search Phase

Your binary search phase was one of the first areas where difficulty began increasing significantly. Problems like Matrix Median and Find Peak Element II felt genuinely hard. What made this phase emotionally difficult was not just the algorithmic complexity — you were also physically sick, mentally exhausted, and trying to maintain consistency.

This led to an important internal conflict. You understood the logic in the moment. But afterward, you feared:

> *"If I opened this again tomorrow, could I actually solve it?"*

Fear of Temporary Understanding

You often worried that your learning was fragile — that you only understood things briefly, that you were forgetting too quickly, that you were creating an illusion of understanding.

However, over time, evidence repeatedly contradicted this fear. When revisiting problems later, concepts often returned quickly, patterns came back faster than expected, reconstruction became easier, and dry runs refreshed understanding rapidly. Your learning was not disappearing — it simply needed retrieval cues.

TREES — THE CORE PHASE

Your Relationship With Trees

Trees became one of the most important phases in your first 60 days. Initially, trees felt intimidating because there were many moving parts, traversal directions felt confusing, iterative implementations looked scary, queues and stacks were unfamiliar, and node pointers felt abstract.

But simultaneously, trees fascinated you. You repeatedly described them as exciting, interesting, visually intuitive, and conceptually satisfying. This combination created a very unique learning experience: you enjoyed the concepts, but the implementation frightened you.

The Diagram vs Code Split

A recurring pattern emerged: you usually understood tree diagrams much faster than tree code. When looking at diagrams, traversal logic made sense, recursion made sense, parent-child relationships made sense, and symmetry made sense.

But during coding, queue front/pop confused you, execution order confused you, pointer usage confused you, helper function parameters confused you, and STL syntax interrupted understanding. Your issue was rarely the algorithm itself — it was often the implementation mechanics.

S Y N T A X & S T L

STL and Syntax Struggles

One of your biggest early weaknesses was unfamiliarity with STL. You had previously mostly worked with vectors, but now you encountered queues, stacks, maps, multisets, pairs, and recursive helper functions.

You frequently struggled with q.front(), q.pop(), q.push(), node->left, node->right, passing vectors/maps by reference, and pointer syntax.

However, an important thing happened — you did not avoid these concepts. Instead you asked questions, requested dry runs, visualized diagrams, revisited execution order, and slowly normalized the syntax. Over time, these stopped feeling magical.

D R Y R U N N I N G

Your Growth Through Dry Runs

Dry running became one of your most powerful learning tools. At first, dry runs were exhausting — you had trouble tracking variables, tracking recursion calls, understanding stack behavior, and understanding queue evolution.

But later, dry runs became the exact thing that unlocked understanding. You repeatedly noticed:

> *"I thought I understood it before, but the dry run made it finally click."*

This became especially important in iterative traversals, queue-based tree problems, vertical order traversal, top/bottom view problems, maximum width problems, and recursive backtracking logic. Dry runs transformed vague understanding into concrete understanding. Over time, you also became more efficient at them.

T H I N K I N G S T Y L E S

Recursive vs Iterative Thinking

One of your most important discoveries was the distinction between recursive thinking and iterative/BFS thinking. You eventually realized: recursion often means solving locally and trusting the call stack, while iterative BFS problems often revolve around queue structure and level management.

You naturally gravitated toward recursion because it matched your intuitive thinking style. You liked local reasoning, focusing on one node, and letting recursion handle repetition. Iterative approaches initially felt like too many moving parts, too much execution-order management, and too much state tracking.

Iterative Traversal Phase

This was one of your biggest struggle periods. Problems like iterative inorder, preorder, and postorder traversal (especially one-stack postorder) caused genuine confusion — not because the idea itself was impossible, but because execution flow was hard to visualize, stack state constantly changed, and syntax and logic interacted simultaneously.

However, something very important eventually happened: you stopped seeing iterative traversal code as random. You began noticing recurring structures, repeated traversal patterns, similar queue logic, and similar conditional structures. This was one of your earliest major pattern-recognition breakthroughs.

PSYCHOLOGICAL GROWTH

One of Your Biggest Psychological Battles

Fear of Passive Learning

Throughout these 60 days, one recurring fear kept appearing: that you were just copying, just following lectures, memorizing without understanding, not truly solving independently.

This fear became strongest when code was long, execution order was complex, you relied on notes, or you checked screenshots frequently.

However, your actual behavior repeatedly contradicted this fear. You consistently reconstructed logic, debugged errors, fixed syntax independently, adapted solutions, recognized patterns, understood dry runs, and explained concepts back to yourself. These are not passive-learning behaviors.

TURNING POINTS

Roman to Integer — An Important Turning Point

The Roman to Integer problem became psychologically important because it was one of the first times you attempted a random LeetCode problem independently, created your own approach, used a map yourself, and debugged implementation mistakes.

Your original approach — storing Roman numeral mappings, iterating through the string, and summing values — was fundamentally correct. The only thing you missed was subtractive notation edge cases.

> *This proved that your intuition was improving. Even when incomplete, you were moving in the correct direction.*

Valid Parentheses — A Different Kind of Lesson

The Valid Parentheses problem affected you differently. Your first instinct involved hash maps and counting characters. But the real solution required stack behavior, order tracking, and structural matching.

This problem taught you something extremely important: data structures are not tied to one topic. Stacks were not "tree-only." This widened your mental flexibility.

T R E E S — P A T T E R N R E C O G N I T I O N

Trees Slowly Became Familiar

One of the biggest transformations across Days 40–60 was this: at first, trees looked chaotic — every problem looked different and implementations felt disconnected. Later, you began recognizing common traversal structures. Queue patterns started repeating. Map usage started repeating. BFS structure became recognizable.

> *Many tree problems are the same skeleton with one key condition changed.*

Examples — top view, bottom view, vertical traversal, and zigzag traversal — all shared similar traversal structures. The only difference was what got stored, when it got overwritten, and how it was grouped. This realization dramatically reduced your fear.

One of Your Most Important Realizations

> *One line of code can completely change the behavior of a problem.*

This was especially visible in top view vs bottom view. That tiny condition — overwrite vs don't overwrite — completely changed the output. Now you were no longer seeing problems as giant unrelated algorithms. You were seeing them as variations of structures, behavioral modifications, and pattern adaptations. This is real algorithmic maturity beginning.

M I N D S E T S H I F T

Your Relationship With Hard Problems

Your reaction to hard problems evolved significantly. Initially, hard tags felt intimidating, long code felt overwhelming, and you doubted yourself quickly. But gradually you stopped panicking at difficulty tags, became curious instead, started attempting before checking solutions, and stayed with confusion longer.

A good example was Vertical Order Traversal. People online described it as extremely difficult. You expected disaster. But afterward, you realized:

> *"I actually understood most of this."*

That moment mattered. It showed your internal standard of difficulty was shifting.

HARDEST MOMENTS

Maximum Width of Binary Tree — One of Your Hardest Days

This problem became one of your biggest struggle moments. You spent hours stuck on overflow/index-reset logic, repeatedly failing to understand the implementation, bouncing between explanations, and becoming mentally exhausted. You nearly quit multiple times.

Your Brain Needs Simplicity First

Complex explanations often made you more confused. You eventually understood the problem only after reducing abstraction, simplifying the dry run, focusing on the core mechanism, and visualizing level-based iteration.

Once you realized the loop runs level-by-level and the minimum index resets each level, everything suddenly clicked. This taught you something very valuable about how you learn.

REVISION STRATEGY

Your Relationship With Revision

Revision became one of your biggest internal battles. You often struggled with rewriting notes repeatedly, lack of novelty, fatigue, boredom, and limited time.

However, over time, your revision strategy evolved. Initially you rewrote everything, repeated code heavily, and depended on handwritten reinforcement. Later, dry runs became more valuable, mental recall became enough sometimes, and selective revision replaced brute-force revision. This was an important maturation.

CONSISTENCY UNDER PRESSURE

Exams vs DSA

A recurring tension throughout this journey was balancing college work, exams, assignments, and DSA consistency. You repeatedly felt exhausted, overloaded, short on time, and guilty for slowing down.

Yet despite this, you continued almost daily — adjusted workload instead of quitting, reduced lecture counts when necessary, and maintained consistency. This adaptability mattered.

CONFIDENCE MOMENTS

Construct Binary Tree Problems — A Confidence Boost

The preorder/inorder and postorder/inorder construction problems became surprisingly enjoyable for you. You liked moving indices, splitting left/right subtrees, the recursion structure, and mapping positions.

This phase showed that your recursive intuition had become much stronger. You were no longer merely following. You were understanding the recursive structure itself.

Serialization and Deserialization

This was one of the first times you encountered stringstream, getline, and serialization logic. Initially the code looked long and intimidating. But you eventually realized the logic itself was understandable — the complexity mostly came from syntax/tools.

This became a recurring theme in your journey: often your real issue was not the algorithm. It was unfamiliar implementation tools.

BST PHASE

Transition Into Binary Search Trees

When you entered BSTs, you immediately felt more comfortable. Binary search intuition returned. You described BSTs as "Binary search in 3D space." This was an excellent intuitive interpretation.

BSTs felt more grounded because ordering existed, navigation had logic, comparisons had meaning, and movement decisions felt natural. This increased your confidence significantly.

Delete Node in BST — A Major Confidence Moment

This was one of your most important confidence-building problems — because you reconstructed most of the code yourself. You understood the helper structure, debugged errors independently, fixed return mistakes, corrected parameter issues, and implemented logic largely from memory.

> *This directly challenged your fear that "Maybe I only understand while watching." Your behavior proved otherwise.*

EMOTIONAL GROWTH

Your Relationship With Errors

Earlier in the journey, errors felt emotionally heavy — bugs felt like proof of incompetence, and syntax mistakes frustrated you intensely. Later, you increasingly saw them as implementation issues, debugging became calmer, and mistakes became learning moments.

Examples included wrong shift operators, wrong indexing, vector return mistakes, passing maps by value instead of reference, and incorrect helper parameters. You slowly stopped treating bugs as identity failures. That is an important emotional shift.

PHASE ONE — STRENGTHS

Your Biggest Strengths (Days 1–60)

1. Self-Reflection

You are exceptionally good at analyzing your own learning. You repeatedly identified where confusion started, whether the issue was syntax or intuition, whether you were learning passively, whether revision was working, and when something genuinely clicked. This self-awareness accelerated your growth.

2. Persistence

Even during sickness, exhaustion, exams, confusion, boredom, and overwhelming problems — you continued. You adjusted pace instead of quitting. That consistency mattered enormously.

3. Pattern Recognition Growth

By Day 60, you had already begun seeing repeated structures, reusable traversal logic, condition-based modifications, and common BFS/DFS skeletons. This was one of your biggest improvements.

4. Recursive Intuition

You became naturally strong at recursive reasoning, subtree thinking, local node solving, and helper function structuring. Recursion suited your brain extremely well.

5. Dry Running Ability

Dry runs evolved into one of your strongest skills. You became capable of mentally simulating execution, spotting behavioral differences, understanding traversal flow, and visualizing recursive returns.

PHASE ONE — WEAKNESSES

Your Biggest Weaknesses (Days 1–60)

1. Fear of Independent Problem Solving

This remained your biggest fear throughout these 60 days. You often worried: "Could I solve this without lectures?" "Am I only reconstructing?" "Would I blank on unseen problems?" This fear was understandable. But your progress increasingly suggested your understanding was real, your reconstruction ability was genuine, and your intuition was developing.

2. Over-Reliance on Immediate Clarification

Sometimes when stuck, especially while tired, you moved toward explanations too quickly. Occasionally, if you had pushed independently slightly longer, you may have discovered the insight yourself. However, you also improved at this over time.

3. Revision Fatigue

You struggled heavily with repetitive rewriting, maintaining enthusiasm for revision, and balancing novelty vs reinforcement. But eventually you adapted your system intelligently.

4. Underestimating Yourself

Repeatedly, your own self-evaluation was harsher than reality. You often believed you understood less than you actually did, were more dependent than you actually were, and were memorizing more than you actually were. But your actual actions showed adaptation, debugging, reconstruction, independent implementation, and conceptual understanding.

PREFERENCES

Your Favorite Things

Across these 60 days, you clearly enjoyed:

- Recursive tree problems
- Constructing trees
- Logical recursive flow
- Elegant boolean recursion
- Discovering hidden patterns
- Realizing two problems share the same skeleton
- Problems where intuition clicks beautifully
- Visual/tree-based reasoning

You especially enjoyed moments where "everything suddenly makes sense."

DEVELOPMENT ARC

Your Development Arc (Days 1–60)

Early Phase (Days 1–30)

- Heavy uncertainty
- Syntax fear
- Implementation confusion
- Little pattern recognition
- Emotional overwhelm

Tree Internalization Phase (Days 30–50)

- Recursion clicks
- BFS/DFS distinction understood
- Traversal structures repeated
- Queue/stack logic normalized
- Dry runs became powerful

Late Tree/BST Phase (Days 50–60)

- Pattern recognition emerging
- Reconstruction improving
- Debugging calmer
- Syntax fear reducing
- Independent coding attempts increasing
- Algorithm families beginning to connect together

Final Assessment at Day 60

By Day 60, you were no longer approaching DSA like a complete beginner. You had already developed recursive intuition, traversal familiarity, BFS/DFS understanding, queue/stack comfort, STL familiarity, debugging resilience, dry-running ability, growing reconstruction skill, and early pattern recognition.

Your biggest remaining weakness was confidence in fully independent problem solving. But importantly, your actual growth already suggested you were capable of much more than you believed.

> *At the beginning, programming often felt like: "How do people even think like this?" By Day 60, your mindset had slowly become: "Okay, this looks familiar. What exactly is changing here?" That is an enormous shift.*

C L O S I N G R E F L E C T I O N — P H A S E O N E

Closing Reflection

The first 60 days were primarily about surviving confusion, building foundations, normalizing recursion, understanding trees, learning STL, recognizing patterns, trusting dry runs, reducing syntax fear, and building consistency.

Most importantly, these 60 days taught you how you personally learn. You learned that intuition matters deeply to you, diagrams help enormously, dry runs unlock understanding, recursion suits your brain, pattern recognition is your strength, long code becomes manageable through familiarity, understanding grows in layers, and confusion is usually temporary.

> *Persistence matters more than perfect confidence. Because many days, confidence was low. But you still continued. And that continuity is what slowly transformed your thinking.*

PHASE TWO

Days 60 – 100

Graphs, Internalization & Algorithmic Maturity

OVERVIEW

100 Days of DSA — Personal Learning Analysis

This section summarizes the DSA journey from roughly Day 60 through Day 100. The purpose is to accurately capture learning patterns, strengths, recurring weaknesses, fears, development over time, methods that worked, methods that slowed progress, and the psychological shifts that happened during the journey.

What stands out most across these 100 days is not simply the number of problems solved. It is the transition from:

- Confusion → Recognition
- Recognition → Internalization
- Internalization → Adaptation

That transformation is very visible in the reports.

LEARNING STYLE

Your Overall Learning Style

You are primarily a pattern-based learner, an explanation-driven learner, a reconstruction-based learner, and an intuition-seeking learner.

You learn best when:

9. You first understand the intuition
10. You watch the implementation
11. You explain it back to yourself
12. You reconstruct the code manually
13. You dry run it
14. You revisit it later

This cycle became your core learning engine. You are not someone who learns effectively through pure memorization. Whenever you tried to memorize mechanically without understanding the intuition, your confidence dropped significantly. However, when intuition clicked, your retention became extremely strong.

PHASE TWO — STRENGTHS

Your Biggest Strengths (Days 60–100)

1. Internalization Through Repetition

This became your greatest strength over time. At the beginning, every problem felt new, every algorithm felt separate, and every implementation felt disconnected. By the graph section, your brain began recognizing reusable structures.

Examples: DFS matrix traversal, BFS traversal, topological sort patterns, Dijkstra template, queue/priority queue mechanics, adjacency list traversal, and distance array updates.

> *You stopped seeing problems as isolated problems. You started seeing templates, patterns, and modifications. This is a major shift.*

2. Extremely Strong Reflective Ability

One of your strongest traits is your ability to analyze your own learning process. You frequently noticed exactly where confusion started, whether the issue was intuition or syntax, whether the issue was implementation or theory, whether the issue was passive watching, whether the issue was exhaustion, and whether revision helped or not.

This level of self-observation is rare. You often accurately identified why a problem felt hard, why a lecture felt confusing, why a dry run suddenly made everything click, and why a template became intuitive. This allowed you to improve your study methods over time instead of staying stuck.

3. Strong Reconstruction Ability

Over time, your ability to reconstruct code improved massively. In the earlier days you relied heavily on screenshots, needed notes frequently, forgot parameters, and forgot traversal patterns. Later you reconstructed full solutions from memory, typed entire solutions without looking, adapted solutions across platforms, rewrote algorithms multiple times in a day, and mentally recalled implementations hours later.

This became especially visible during Topological Sort / Kahn's Algorithm, Course Schedule I & II, Dijkstra-based problems, and Word Ladder problems.

4. Adaptation Ability

A very important development was your ability to adapt one platform's solution to another platform's problem statement. Examples: GeeksforGeeks to LeetCode conversions, changing 1-based conditions to 0-based, changing source/destination assumptions, modifying matrix traversal conditions, and adjusting graph representations.

> *This is a major step beyond simple copying. It showed that you were beginning to understand the underlying structure rather than only memorizing surface syntax.*

5. Persistence and Consistency

Even during exhaustion, exams, low motivation, heat, travel, college fatigue, and chaotic schedules — you continued doing DSA almost every day. Sometimes you reduced the workload. Sometimes you combined reports. Sometimes you skipped revision. But you rarely stopped completely. That consistency mattered enormously.

6. Increasing Debugging Confidence

Earlier mistakes often felt emotionally heavy. Later, mistakes became "just typos," "just overflow issues," "forgot long long," "wrong condition." This emotional shift is important. You stopped treating every bug as proof that you did not understand the problem. That is a sign of increasing programming maturity.

PHASE TWO — WEAKNESSES

Your Biggest Weaknesses (Days 60–100)

---

1. Weak Independent Problem Discovery

This remained your biggest weakness throughout most of the journey. You became very good at understanding after explanation, reconstructing after watching, and adapting after learning. But you often feared being given a completely fresh problem, not recognizing the correct algorithm, not knowing where to start, and drawing a blank without a lecture.

This fear was real. However, by Day 100, evidence suggested you were stronger than you believed. You already showed signs of independent thinking when adapting problems, modifying templates, changing constraints, identifying patterns yourself, and making partial implementations before correction.

Codeforces contests were a very good addition specifically because they target this weakness.

2. Passive Consumption During New Algorithms

When an algorithm was completely new, you sometimes shifted into passive watching mode. This happened especially with Floyd-Warshall, Bellman-Ford, some topological sort concepts, and complex graph state problems. The good news: you became increasingly aware of this issue over time.

3. Fear of Long Code

Large codebases initially overwhelmed you. You often described reactions like "oh my god this is huge" or "why is this so long?" — feeling mentally blocked by screenshots.

Over time, this improved significantly. You eventually realized most graph problems looked long because templates repeated, DFS/BFS helpers repeated, traversal logic repeated, and boundary checks repeated. Once templates became internalized, long code stopped feeling intimidating.

4. Inconsistent Revision Motivation

You repeatedly struggled with revision more than learning new problems. Reasons included repetition fatigue, boredom, notebook-writing exhaustion, environmental fatigue, and lack of novelty. However, you eventually evolved your revision system intelligently by realizing not every problem needed full notebook rewriting, dry runs often gave more value, and mental recall could replace brute-force rewriting.

5. Problem Statement Interpretation

You frequently struggled with overly worded problem statements, abstract graph wording, and platform-specific phrasing — especially on graph problems, path/counting problems, and shortest path variants. However, you developed a strategy: create your own interpretation first, then simplify mentally before watching. This strategy helped significantly.

RECURRING FEARS

Your Main Fears

Fear of "Being Nothing Without Lectures"

This became one of your deepest recurring concerns. You worried that without Striver, without explanations, without guided patterns — you might fail completely. This fear decreased over time but never fully disappeared.

However, your actual behavior contradicted this fear increasingly often. You already demonstrated adaptation, reconstruction, debugging, platform conversion, and partial independent reasoning. These are not the abilities of someone who only memorizes.

Fear of Blank Mind Syndrome

You often feared opening a fresh problem, having no idea where to start, not recognizing the algorithm, and not generating the correct conditions. This is common among developing programmers. Your solution to this fear eventually became Codeforces contests, trying adaptations yourself, forcing reconstruction, and doing dry runs.

Fear of Forgetting

Early on, you feared forgetting heavily. This led to extensive notebook rewriting, repeated revision systems, screenshot collection, and aggressive recall attempts. Over time, your confidence improved because templates stayed in memory naturally, reconstruction became easier, and repeated exposure created long-term familiarity.

HONEST REFLECTION

What You Did Wrong

1. Sometimes Over-Reliance on Immediate Explanation

In some moments, especially when tired, you moved too quickly toward explanations. The best example was Longest Common Prefix — you had a partial intuition already. Instead of pushing slightly further independently, you looked at code too early. You recognized this mistake yourself afterward.

2. Underestimating Your Own Ability

Repeatedly, you assumed "I probably couldn't do this alone." But your actions increasingly showed otherwise. You often got close independently, produced nearly correct logic, adapted code successfully, and solved syntax rather than conceptual issues. You frequently judged yourself more harshly than the evidence supported.

3. Using Revision Methods Past Their Peak Usefulness

Your original notebook-heavy system worked very well initially. But later, many graph problems became template variants, rewriting everything became repetitive, and cognitive value decreased. Eventually you noticed this yourself and began evolving the system. That was the correct move.

PROGRESS OVER TIME

What Became Easier Over Time

Graph Traversals

Initially DFS/BFS felt mechanically difficult, parameter lists felt overwhelming, and delRow/delCol patterns required notes. Later, traversal became automatic, matrix movement became intuitive, and helper functions became natural.

Template Recognition

This may be your single biggest improvement. You increasingly categorized problems into BFS template, DFS template, Dijkstra template, topological sort template, and shortest path template. That recognition accelerated learning massively.

Dry Running

Earlier dry runs were mentally exhausting. Later you simplified variable tracking, stopped over-tracking unnecessary values, followed execution flow naturally, and began enjoying dry runs. Dry runs eventually became one of your strongest learning tools.

Debugging

You became calmer around syntax mistakes, declaration mistakes, wrong conditions, overflow problems, and submission issues. This emotional stabilization matters a lot.

REMAINING CHALLENGES

What Stayed Hard For You

Brand-New Algorithms

Every time a genuinely new algorithm appeared, your confidence dipped temporarily. Examples: Bellman-Ford, Floyd-Warshall. Not because the code was hard — usually because the intuition layer was unfamiliar, the philosophy changed, or the mental model shifted. However, you adapted faster each time.

Multi-State Graph Problems

Problems with many moving parts confused you initially. Examples: Word Ladder II, Eventual Safe States, and problems with multiple arrays/vectors. You often needed dry runs to fully consolidate understanding.

Ambiguous Problem Statements

Even by Day 100, long problem statements still annoyed and slowed you. But your interpretation strategy improved this significantly.

DEVELOPMENT TIMELINE

Your Development Over Time

Early Phase (Days 1–40)

Likely characterized by overwhelm, syntax dependency, weak template recognition, strong uncertainty, and a feeling that every problem was unique.

Middle Phase (Days 40–70)

This is where internalization began. You started recognizing patterns, remembering structures, reconstructing helper functions, and understanding repeated logic. Confidence slowly increased.

Graph Internalization Phase (Days 70–100)

This was your biggest leap. You developed adjacency list familiarity, graph traversal intuition, shortest path familiarity, topological sort understanding, multi-array state management, and strong template recall. This phase also introduced adaptation ability, independent modifications, more confidence in debugging, and deeper self-analysis.

PSYCHOLOGICAL SHIFTS

Important Psychological Shifts

Shift 1: From Memorization to Pattern Recognition

You stopped treating every problem as isolated. This was foundational.

Shift 2: From Fear of Code to Familiarity With Structures

Long graph code stopped feeling impossible. You realized most of it was repeated structure — only small sections changed.

Shift 3: From Syntax Anxiety to Logical Thinking

Earlier, syntax errors felt catastrophic. Later, logic mattered more and syntax became adjustable.

Shift 4: From External Dependence to Self-Correction

Earlier, confusion often required external help immediately. Later, you isolated confusion yourself, identified missing intuition layers, and repaired understanding actively. This is one of the strongest developments in your journey.

FINAL ASSESSMENT

Day 100

Where You Stand — and Where You're Headed

DAY 100 ASSESSMENT

Final Assessment at Day 100

At Day 100, you are no longer a complete beginner. You are somewhere in the transition between beginner and lower intermediate.

Your strongest area is currently:

- Graph template understanding
- Reconstruction
- Algorithm adaptation
- Implementation recall

Your weakest area is currently:

- Independent first-principles problem solving
- Solving unseen problems fully alone

However, you already possess many foundational skills necessary to improve this:

- Pattern recognition
- Debugging
- Dry running
- Implementation fluency
- Algorithm familiarity

Your progress is very real. The most important development is probably this:

> *You no longer sound intimidated by programming itself. Even when confused, your mindset has increasingly become: "What exactly am I missing here?" instead of "I can't do this." That shift matters enormously.*

LOOKING AHEAD

The Next Phase

The first 100 days were primarily about building foundations, internalizing patterns, learning algorithm families, becoming comfortable with graph thinking, and learning how you personally learn best.

The next phase will likely be more about:

- Independent problem solving
- Reducing lecture dependence
- Strengthening intuition generation
- Solving unseen problems
- Competitive programming growth
- Deeper algorithmic thinking

> *The good news is: you now possess enough structure internally that these goals are realistic. The internalization phase has already begun.*